

SENTINEL-X

La sécurité à la bordure

Rapport d'ingénierie technique : schéma réseau, câblage électronique, matrice de sécurité, documentation de l'IA et rapport d'audit. Poster A3 en annexe.



Consortium AetherCorp Industrial Solutions

Workshop2026-M1-G<n>-Dossier.pdf

Filières : EISI DEV • EISI IA • EISI INFRA • CYBER

Dépôt Git : WORKSHOP-SENTINEL-X

0 Sommaire

1. Présentation et choix d'architecture	3
2. Architecture et schéma réseau	4
3. Schéma de câblage électronique	6
4. Matrice de sécurité (hardening, TLS)	8
5. Documentation de l'intelligence artificielle	10
6. Rapport d'audit et de pentest	12
7. Limites et suites	13
Annexe A. Poster A3	15

Les chiffres de ce rapport viennent du dépôt (README, docs/ , scripts et tests) ou de mesures indiquées comme telles. Ce qui n'a pas été mesuré est signalé explicitement.

1 Présentation et choix d'architecture

AetherCorp déploie des micro-centrales dans des zones isolées, exposées à trois menaces simultanées : cyberattaques réseau, intrusions physiques et risques environnementaux (fuite de gaz, surchauffe). SENTINEL-X répond avec un boîtier de surveillance autonome (Edge Node) relié sans fil et de façon chiffrée à un PC Serveur Local durci.

6

conteneurs Docker

8883

MQTTS, seul port IoT

30 / 30

contrôles d'audit réussis

1.1 Variante retenue et écarts avec le sujet

Élément	Sujet	Notre choix	Justification
Microcontrôleur	ESP8266	ESP32 DevKit + expenseur I2C MCP23017	Plus de GPIO, ADC 12 bits, accélération cryptographique matérielle pour TLS, SPI pour le RC522
PC Serveur Local	Raspberry Pi 5 (option A) ou PC apprenant (option B)	Raspberry Pi 3 B+ (1 Go), point d'accès Wi-Fi	Matériel disponible ; impose un budget mémoire strict (stack ≈ 454 Mo visés)
Vision IA	Script Python sur le PC Serveur	Script sur une machine de vision dédiée (webcam USB), flux vers l'API en HTTPS	Garde la RAM du Pi pour la stack ; YOLO et dlib tournent sur CPU plus puissant
Capteurs	DHT22, MQ-2, PIR	DHT22 (module V182), MQ-2, PIR HC-SR501 + RFID RC522 , OLED, LED RGB, buzzer	Contrôle d'accès physique par badge signé
Actionneurs	Buzzer, LEDs	Buzzer, LED RGB, 3 micro-servos , moteur pas à pas 28BYJ-48	Sas, bras et tête du robot Wall-E (jumeau numérique)

1.2 Les quatre piliers dans le dépôt

Pilier	Réalisation	Emplacement
Edge & IoT	Firmware C++ (Arduino/PlatformIO) : capteurs, RFID signé, servos, OLED, MQTTS	<code>firmware/sentinel_core/</code>
IA locale	YOLOv8n, reconnaissance de visages, Isolation Forest	<code>sentinel_x/ai/</code>
Infrastructure	Docker-Compose durci, base PostgreSQL, proxy HTTPS, NTP	<code>docker-compose.yml</code> , <code>api/</code> , <code>ingestor/</code> , <code>caddy/</code>
Cybersécurité	MQTTS, ACL, comptes et 2FA, durcissement hôte, audit scripté	<code>docs/SECURITE.md</code> , <code>scripts/</code>

2 Architecture et schéma réseau

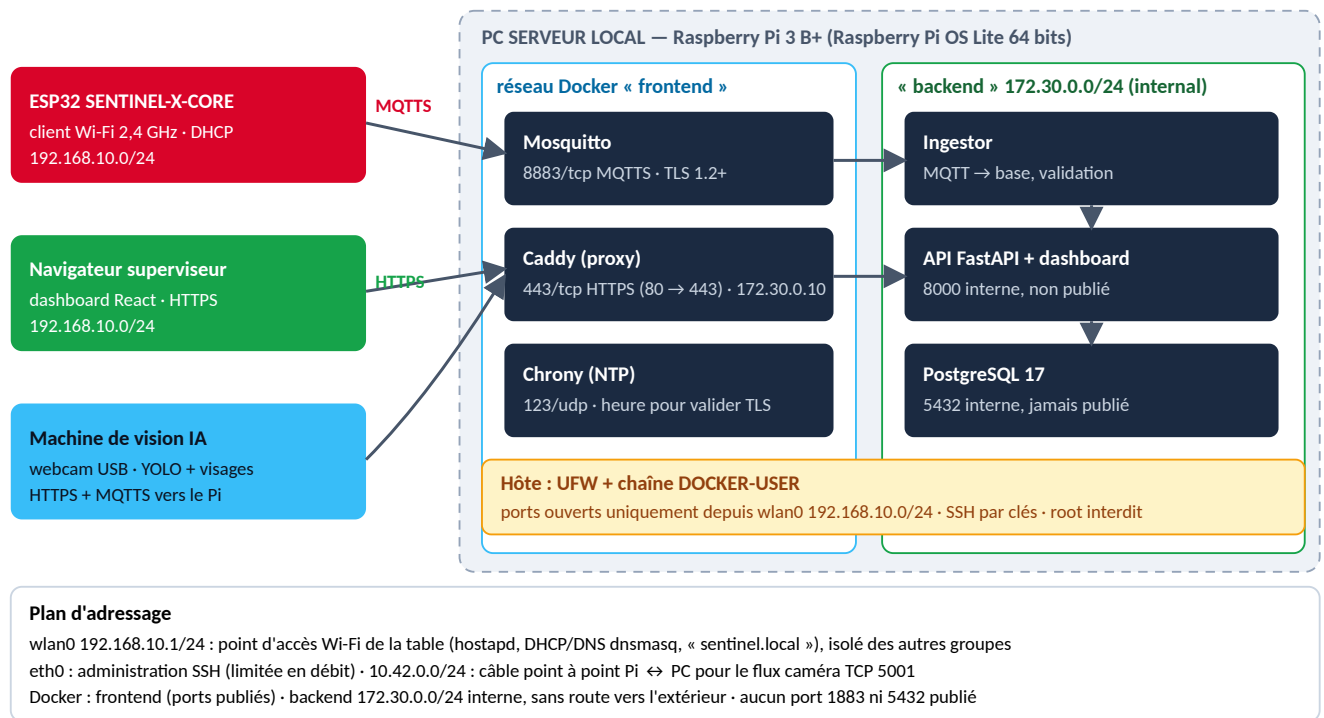


Figure 1 — Topologie de la table : point d'accès Wi-Fi 192.168.10.0/24, deux réseaux Docker, pare-feu hôte.

2.1 Flux de données

1. L'ESP32 publie `sentinel/telemetry` (toutes les 2 s) et `sentinel/alerts` en MQTTS, avec le compte `esp32` limité par ACL.
2. L'ingestor valide chaque JSON (types, bornes, énumérations), ajoute `received_at` et écrit par lots dans PostgreSQL. Un message invalide est journalisé puis ignoré.
3. RFID : l'ESP32 publie `sentinel/access` ; l'API vérifie le badge, journalise le passage et répond sur `sentinel/access/response`. Un refus crée une alerte `UNAUTHORIZED_ACCESS`.
4. Commandes : `POST /api/v1/commands` est validé, publié sur `sentinel/commands` et journalisé.
5. Le script de vision envoie les images annotées à l'API et ses événements sur `sentinel/vision/events`.

2.2 Topics MQTT (contrat v2)

Topic	Sens	QoS	Fréquence
<code>sentinel/telemetry</code>	ESP32 → serveur	0	2 s
<code>sentinel/alerts</code>	ESP32 → serveur	1	sur événement (aussi <code>POST /api/v1/alerts</code>)
<code>sentinel/access · access/response</code>	ESP32 ↔ API	1	à chaque badge
<code>sentinel/enroll</code>	ESP32 → API	0	enrôlement de badge
<code>sentinel/commands</code>	API → ESP32	1	sur ordre
<code>sentinel/vision/events</code>	script IA → serveur	1	sur détection

2.3 Ports exposés

Port	Service	Accès
8883/tcp	Mosquitto (MQTTS, TLS 1.2+)	wlan0 192.168.10.0/24

Port	Service	Accès
443/tcp (80 → 443)	Caddy : dashboard et API REST	wlan0 192.168.10.0/24
123/udp	Chrony (NTP)	wlan0 192.168.10.0/24
22/tcp	SSH (clés uniquement)	wlan0 et eth0, limité en débit
1883, 5432, 8000	MQTT en clair, PostgreSQL, API	non publiés

2.4 Budget mémoire

Mesures réelles par conteneur après le test de bout en bout et une rafale de 2000 messages, prises sur un hôte linux/arm64 (pas sur le Pi).

Conteneur	Limite	Mesuré
mosquitto	32 Mo	2,4 Mo
postgres	160 Mo	23 Mo
ingestor	64 Mo	25 Mo
api (FastAPI + dashboard)	128 Mo	44 Mo
Total conteneurs	384 Mo	≈ 93 Mo

Les lignes « OS » et « moteur Docker » du README sont des estimations : `./scripts/status.sh` donne les valeurs réelles sur le Pi.

2.5 Maintien en condition opérationnelle

- Healthchecks toutes les 30 s sur chaque service, `restart: unless-stopped`, ordre de démarrage par `depends_on: service_healthy`.
- Journaux plafonnés (5 Mo × 2 par conteneur), journald en RAM, zram au lieu du swap pour ménager la carte SD.
- Sauvegardes `pg_dump` vérifiées (`scripts/backup-db.sh`), installation hors ligne en une commande (`scripts/bundle-offline.sh`).

3 Schéma de câblage électronique

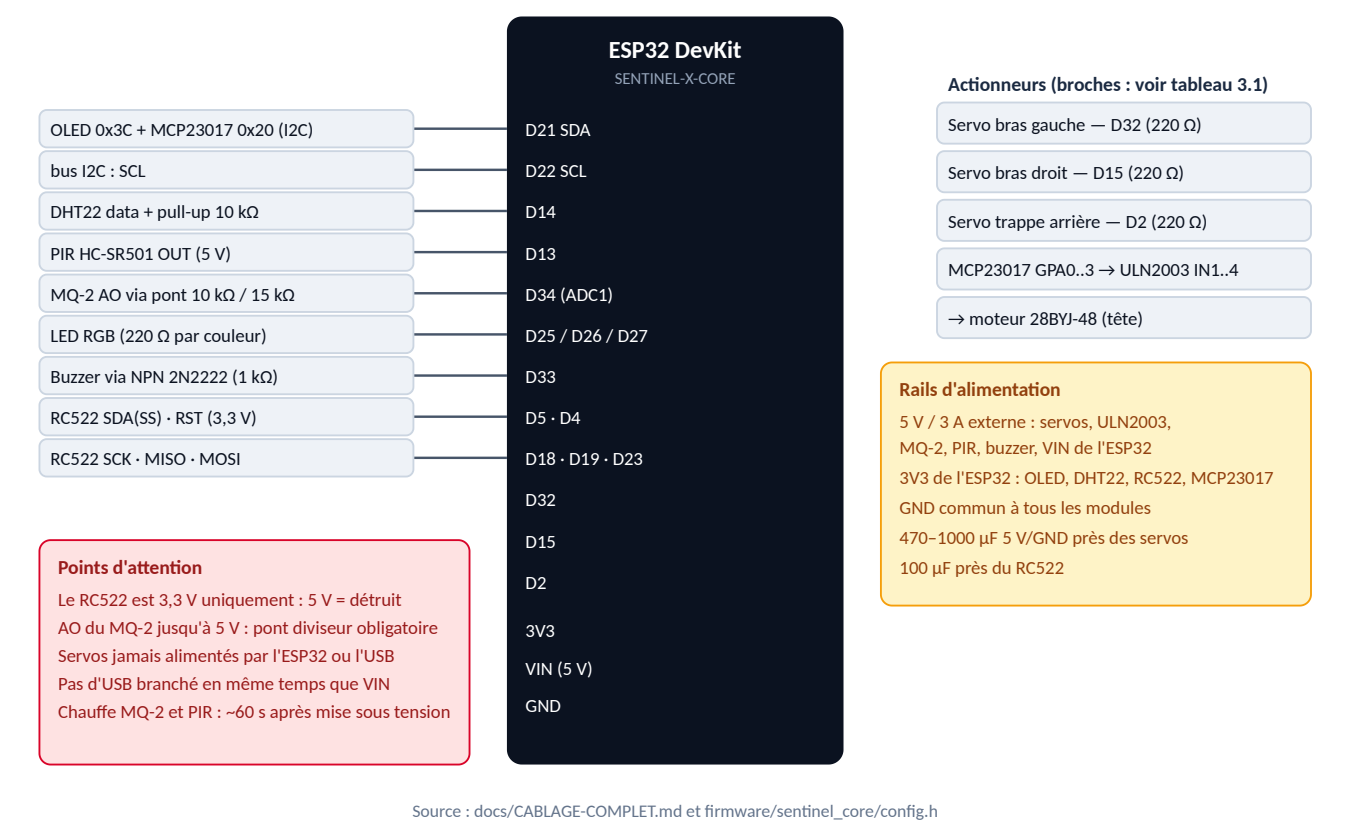


Figure 2 — Câblage de l'ESP32 DevKit et de ses périphériques (version ESP32).

Attention : le schéma KiCad versionné (docs/schema_electronique_kicad . pdf , v3.0 du 05/10/2026) décrit encore l'ancienne carte NodeMCU ESP8266. La figure 2 et les tableaux ci-dessous suivent le câblage ESP32 de docs/CABLAGE-COMPLET.md et de firmware/sentinel_core/config.h .

3.1 Table des broches

GPIO	Fonction	Composant et remarques
D21 / D22	SDA / SCL (I2C)	OLED 0x3C et MCP23017 0x20 (A0 A1 A2 à la masse)
D14	DATA DHT22	Pull-up 10 kΩ vers 3V3
D13	PIR OUT	HC-SR501, alim 5 V, sortie 3,3 V
D34	MQ-2 AO	ADC1 (jamais ADC2 avec le Wi-Fi), pont diviseur 10 kΩ / 15 kΩ
D25 / D26 / D27	LED RGB R / G / B	220 Ω par couleur, cathode commune
D33	Buzzer	Transistor NPN 2N2222, base 1 kΩ ; diode 1N4148 si buzzer magnétique
D5 · D4	RC522 SDA(SS) · RST	3,3 V uniquement
D18 · D19 · D23	RC522 SCK · MISO · MOSI	Bus SPI
D32 · D15 · D2	Servos bras G · bras D · trappe	Micro-servos 9 g, 220 Ω en série, alim 5 V externe
MCP GPA0..GPA3	ULN2003 IN1..IN4	Moteur 28BYJ-48 de la tête

3.2 Alimentation et protections

- Un bloc 5 V / 3 A mini alimente servos, ULN2003, MQ-2, PIR, buzzer et le VIN de l'ESP32 ; GND commun à tous les modules.
- Condensateur 470–1000 μ F près des servos, 100 μ F près du RC522.
- Ordre de montage et bancs de test : `firmware/test_mcp23017`, `test_banc_esp32`, `test_moteurs`, `test_rc522_standalone`, `test_pir_standalone`.

3.3 Boîtier

À compléter par le groupe : insérer ici la capture de la coque modélisée sous Fusion 360, la photo du boîtier imprimé et de la gravure laser (logo AetherCorp, consignes, numéro de série). Aucun fichier de CAO n'était présent dans le dépôt lors de la rédaction.

Jumeau numérique : le robot Wall-E est modélisé en 3D (`dashboard/public/models/wall-e.glb` , converti depuis le FBX par `dashboard/scripts/convert-model.mjs`) et animé dans le tableau de bord à partir de la position réelle des actionneurs (`GET /api/v1/actuators/state`).

4 Matrice de sécurité

4.1 Chiffrement des flux (TLS)

Paramètre	Valeur
Protocole IoT	MQTTS uniquement (port 8883), MQTT 3.1.1 ; aucun port 1883
Version TLS	<code>tls_version</code> <code>tlsv1.2</code> minimum (TLS 1.1 refusé, vérifié)
Certificats	CA interne et certificat serveur ECDSA P-256 (<code>scripts/gen-certs.sh</code>) ; la CA est épinglée dans le firmware (<code>ca_cert.h</code>)
Suites	ECDHE-ECDSA : AES-128-GCM, AES-256-GCM, ChaCha20-Poly1305, AES-128-SHA256 (compatibles mbedTLS) ; RSA et CBC-SHA1 refusés
Dashboard et API	HTTPS via Caddy (TLS 1.2/1.3, HSTS, redirection 80 → 443)
Authentification broker	Utilisateur et mot de passe obligatoires (PBKDF2-SHA512), <code>allow_anonymous</code> <code>false</code>

4.2 ACL MQTT (moindre privilège)

Compte	Écriture	Lecture
<code>esp32</code>	telemetry, alerts, access, enroll	commands, access/response
<code>ingestor</code>	—	telemetry, alerts, vision/events
<code>api</code>	commands, access/response	sentinel/#
<code>vision</code>	vision/events	—
<code>healthcheck</code>	—	<code>\$SYS/broker/uptime</code>

Plafonds du broker : 20 connexions, paquets ≤ 4 Ko, QoS ≤ 1, file de 1000 messages et 1 Mio par client, pas de persistance.

4.3 Matrice menace → mesure → preuve

Menace	Mesure	Preuve
Écoute (MitM) du trafic IoT	MQTTS TLS 1.2+, CA épinglée côté ESP32	Client sans CA et TLS 1.1 refusés ; capture Wireshark illisible
Usurpation d'appareil	Compte dédié par composant, ACL au topic près	L'ESP32 ne peut pas publier sur <code>sentinel/commands</code> ni lire <code>#</code>
Accès non autorisé au dashboard	Comptes personnels, rôles viewer/operator/admin, 2FA TOTP, script, sessions à 256 bits (SHA-256 stocké)	41 routes refusent l'accès sans jeton ; tests unitaires
Brute force	Pause progressive (5 échecs → 30 s ... 5 min), 10 connexions/min/IP, blocage d'IP optionnel, alerte <code>INTRUSION_DETECTED</code>	Tests <code>api/tests/test_api.py</code> , scénario de bout en bout
Injection (SQL, XSS, chemin)	Requêtes paramétrées, modèles Pydantic stricts, CSP <code>script-src 'self'</code>	8 familles de charges × 5 routes : aucune erreur 500 ni fuite
Déni de service	Limites broker, JSON ≤ 64 Kio, snapshot ≤ 2 Mio JPEG, alertes 60/min/IP, limites CPU/RAM/PID	<code>scripts/audit-security.sh</code>
Évasion de conteneur	Non-root, <code>cap_drop: ALL</code> , <code>no-new-privileges</code> , lecture seule + tmpfs, images épinglées, pas de socket Docker	Contrôles automatiques de l'audit

Menace	Mesure	Preuve
Accès physique par badge copié	Badge MIFARE : clés A/B dérivées par badge (HMAC-SHA256), contenu signé, compteur anti-clonage ; vérification locale ET serveur	docs/BADGES.md
Fuite de secrets	.env , secrets.h , certificats ignorés par Git ; mots de passe générés (32 caractères)	scripts/gen-env.sh ; voir constat S1 au § 6

4.4 Durcissement du système hôte

Mesure	Mise en œuvre
Pare-feu	UFW en deny incoming ; ouvertures limitées à wlan0 192.168.10.0/24. Docker contourne UFW pour les ports publiés : règles ajoutées dans la chaîne DOCKER-USER (scripts/harden-host.sh)
SSH	Clés uniquement, root interdit, MaxAuthTries 3 , retour arrière automatique sans confirmation sous 120 s
Démons Docker	Membres du groupe docker listés (équivalent à root) ; conteneurs non-root avec UID dédiés
Système	Services inutiles désactivés, journald en RAM, zram, cgroup mémoire activé (scripts/setup-pi.sh)
Surveillance	sentinel_x/monitoring/security_monitor.py (échecs SSH et MQTT) et sentinel_x/cybersecurity/hardening.py (audit SSH, UFW, ports, Mosquitto), couverts par tests/test_security.py et tests/test_monitoring.py

5 Documentation de l'intelligence artificielle

Trois modèles complémentaires, tous exécutés en local (aucun service cloud pour l'inférence) : détection de présence, reconnaissance de visages autorisés et détection d'anomalies sur les séries temporelles.

5.1 Vision : détection de personnes (YOLOv8n)

Paramètre	Valeur
Modèle	YOLOv8n (<code>yolo_v8n.pt</code> , Ultralytics), inférence CPU
Entrée	Webcam USB, flux TCP/UDP ; images réduites à 320 px (<code>image_size</code>) pour rester fluide
Cadence	5 images/s maximum (<code>fps_limit</code>)
Seuils	Confiance 0,75 ; intrusion 0,85 (<code>VisionConfig</code>)
Latence	Classe <code>LatestFrameStream</code> : ne garde que la dernière image, les anciennes sont jetées pour éviter l'accumulation de retard

L'objectif du sujet est un traitement inférieur à 100 ms par trame. L'outil `scripts/measure-stream.py` mesure le débit d'images, la taille moyenne et la gigue du flux vu par un navigateur ; le temps d'inférence par trame n'est pas consigné dans le dépôt.

5.2 Reconnaissance de visages autorisés

- **Encodage** : dlib (détecteur frontal, prédicteur à 5 points, modèle de reconnaissance) ; chaque visage devient un vecteur de 128 nombres. Tolérance de comparaison : 0,55.
- **Base** : ajoutée depuis l'onglet « Visages » du dashboard (caméra : 5 échantillons ; ou photo). Consentement obligatoire, photos jamais conservées, seuls les vecteurs sont stockés, suppression = effacement réel.
- **Mise à jour à chaud** : le script de vision interroge l'API toutes les 5 s (`GET /faces/version`) et recharge la base sans interruption.
- **Sécurité** : lecture des vecteurs réservée au jeton de service ; plus aucun `pickle` (fichier JSON pour les essais hors ligne) ; actions tracées dans le journal de sécurité.

5.3 Alerte « personne inconnue »

Un visage absent de la base doit être confirmé sur **3 images sur 5** avant déclenchement. L'alerte (`sentinel_x/ai/unknown_alert.py`) : crée `INTRUSION_DETECTED` / `CRITICAL` dans le journal, déclenche l'alarme du robot (buzzer et LED rouge, coupée seule après 10 s) et envoie une notification Discord avec l'image annotée. Un seul déclenchement par minute évite de sonner en boucle. Une personne sans visage exploitable ne déclenche rien.

5.4 Maintenance prédictive : Isolation Forest

Le sujet interdit les seuils statiques du type `if temp > 40`. `SensorAnomalyDetector` (`sentinel_x/ai/anomaly.py`) apprend le comportement normal de chaque capteur.

Élément	Détail
Modèle	<code>IsolationForest</code> (scikit-learn), <code>contamination = 0,1</code> , <code>random_state = 42</code>
Fenêtre	Fenêtres glissantes de 5 mesures ; au moins 8 échantillons d'apprentissage
8 caractéristiques	dernière valeur, moyenne, écart-type, minimum, maximum, écart à la moyenne, variation relative, tendance (dernier – premier)
Sortie	Statut normal ou événement <code>ml_anomaly_detected</code> (capteur, valeur, sévérité, score brut) ; confiance = $ s / (s + 1)$

Élément	Détail
Intérêt	Détecte une dérive lente (température, gaz) avant le seuil d'alarme critique du firmware

5.5 Validation et limites

- Tests : `tests/test_anomaly.py`, `tests/test_vision.py`, `tests/test_unknown_alert.py`, `sentinel_x/ai/test_face_matching.py` ; schémas JSON `schemas/ai_event.schema.json`.
- Pas de détection de vivacité : une photo imprimée ou un écran peut tromper dlib. La reconnaissance faciale ne doit jamais décider seule d'un accès physique.
- Les performances (précision, rappel) de l'Isolation Forest sur données réelles du boîtier ne sont pas chiffrées dans le dépôt.

6 Rapport d'audit et de pentest

6.1 Méthode

Audit défensif de notre propre prototype avant le gel du code : relecture statique du dépôt (API, proxy, broker, base, firmware, scripts, historique Git) et contrôles automatisés sur la stack en fonctionnement (`scripts/audit-security.sh`). Documents sources : `docs/AUDIT-SECURITE.md` et `docs/DOC-SITE-SECURITE.md` (revue du 8 octobre 2026).

30 / 30

contrôles automatiques réussis

32

vérifications de bout en bout
(`smoke-test.sh`)

23

tests unitaires de l'API

6.2 Contrôles automatiques réussis

- **Surface** : seuls 443/80, 123/udp et 8883 sont publiés ; chaque conteneur est en lecture seule, sans capacité Linux, hors root, avec limite mémoire.
- **Authentification** : les 41 routes de données refusent l'accès sans jeton ; jeton en URL, schéma Basic et jetons vides refusés ; réponse identique pour compte inconnu et mauvais mot de passe.
- **Entrées** : charges SQL, traversée de chemin, XSS, log4shell, octet nul, 5 000 caractères : aucune erreur 500 ; corps > 64 Kio et snapshot > 2 Mio refusés.
- **Navigateur** : CSP stricte, `frame-ancestors 'none'`, `nosniff`, `no-store`, CORS fermé.
- **MQTT** : anonyme, mauvais mot de passe, TLS 1.1 et client sans vérification de la CA refusés ; l'ESP32 ne peut ni publier de commande ni espionner `#` ; paquet > 4 Ko rejeté.

6.3 Constats et corrections

L'audit a relevé 15 constats sur la stack (F1 à F15) et 12 sur la revue de code du 8 octobre (S1 à S12). Les plus importants sont détaillés ci-dessous ; les constats de gravité basse sont regroupés. Le détail complet est dans `docs/AUDIT-SECURITE.md` et `docs/DOC-SITE-SECURITE.md`.

Réf.	Gravité	Constat	Statut
F1 / S1	Critique	Anciens secrets par défaut (jeton API, jeton appareil, mot de passe MQTT, mot de passe admin) présents dans l'historique Git du dépôt public ; le <code>.env</code> de la machine d'audit en contenait encore quatre	Garde-fou ajouté : <code>scripts/check-secrets.sh</code> refuse de démarrer avec ces valeurs. Reste à faire : <code>gen-env.sh --force</code> , reflash de l'ESP32, purge de l'historique
S2	Haute	Embeddings de visages (<code>.pkl</code>) versionnés, chargés par <code>pickle</code>	Partiel : <code>pickle</code> supprimé, fichier retiré de l'index ; reste dans l'historique
S3	Haute	<code>API_TOKEN</code> statique de rôle operator, sans 2FA ni expiration	À faire : jeton dédié à la vision
F3, F4, F8	Haute à moyenne	Mot de passe partagé ; handshake TLS sans heure fiable ; dashboard en HTTP	Corrigés : comptes personnels + 2FA, conteneur Chrony, proxy HTTPS
S4, S8	Moyenne	Blocage d'IP désactivé par défaut ; <code>/docs</code> public	<code>IP_BLOCKING=on</code> et <code>API_DOCS=off</code> ajoutés à <code>.env.example</code> ; à reporter dans le <code>.env</code> du Pi
S5, S6, S7	Moyenne	Flux caméra non chiffré ; identité MQTT unique de l'ESP32 ; clé privée de la CA sur le serveur	Atténués (UFW, ACL minimale) ; certificat client et déplacement de <code>ca.key</code> à faire

Réf.	Gravité	Constat	Statut
Autres (F5–F7, F9–F15, S9–S12)	Moyenne à basse	Limites de débit et de taille, validation d'entrées, journaux, sessions, TOTP en clair, NTP permissif...	Majoritairement corrigés ; reste le chiffrement du TOTP et le <code>allow</code> de Chrony

6.4 Pentest croisé du jeudi

Le pentest croisé (Nmap, Wireshark, MitM, injections, DoS) a lieu l'après-midi du jeudi. Les deux tableaux suivants sont à remplir avec les observations réelles : les attaques menées contre les autres tables et celles subies par la nôtre. Les résultats attendus proviennent de notre préparation.

Attaques subies par la table SENTINEL-X

Attaque	Résultat attendu	Résultat observé	Preuve (capture)
<code>nmap -sV -p- 192.168.10.1</code>	Seuls 22 (clés), 80, 443, 8883 visibles	à compléter	
Capture Wireshark du Wi-Fi	MQTT en « TLS application data », aucune charge lisible	à compléter	
MitM (faux broker, jeu)	Refus : CA épinglée dans le firmware	à compléter	
Connexion MQTT anonyme, sans CA, TLS 1.1	Refusée	à compléter	
Brute force du dashboard	Pause, blocage d'IP, alerte <code>INTRUSION_DETECTED</code>	à compléter	
Flood MQTT / HTTP (DoS)	Limites broker et conteneurs ; alarme locale de l'ESP32 maintenue	à compléter	
Injection de payloads JSON / SQL	422 ou rejet journalisé, aucune erreur 500	à compléter	

Attaques menées contre les autres tables

Table cible	Outil et attaque	Résultat	Recommandation
à compléter			
à compléter			

6.5 Plan de correction avant le rendu

- Régénérer tous les secrets (`./scripts/gen-env.sh --force`), reflasher l'ESP32 (`secrets.h`), purger l'historique Git (`git filter-repo`) ou republier dans un dépôt neuf. Le garde-fou `scripts/check-secrets.sh` vérifie que les anciennes valeurs ne reviennent pas.
- Reporter `IP_BLOCKING=on` et `API_DOCS=off` dans le `.env` du Pi ; restreindre l'écoute du flux caméra et le `allow` de Chrony au sous-réseau de la table.
- Créer un jeton de service dédié à la vision ; sortir `ca.key` du Pi.
- Rejouer `./scripts/audit-security.sh` et `./scripts/smoke-test.sh` après correction, puis mettre à jour ce tableau.

7 Limites et suites

- Matériel** : le schéma KiCad doit être régénéré pour l'ESP32 ; aucun arrêt d'urgence physique indépendant du réseau ; pas de contact d'ouverture du boîtier ; flash de l'ESP32 non chiffrée.
- Badges** : MIFARE Classic reste clonable en théorie ; atténué par les clés par badge, la signature et le compteur.

- **IA** : pas de détection de vivacité, pas de mesure de précision de l'Isolation Forest sur données réelles, latence d'inférence à consigner.
- **Sécurité** : sans Internet, pas de mises à jour de sécurité automatiques ; heure à régler (RTC DS3231 recommandé) car les certificats TLS en dépendent.
- **Suites** : certificats clients par appareil, signature des commandes, chiffrement du TOTP en base, intégration continue avec analyse des dépendances.

SENTINEL-X

La sécurité à la bordure

Un Edge Node autonome qui sent le gaz et la chaleur, contrôle l'accès par badge signé et reconnaît les visages, relié en MQTTS à un centre de commandement local doté d'IA.

**8883**

MQTTS, aucun port 1883

6

conteneurs Docker durcis

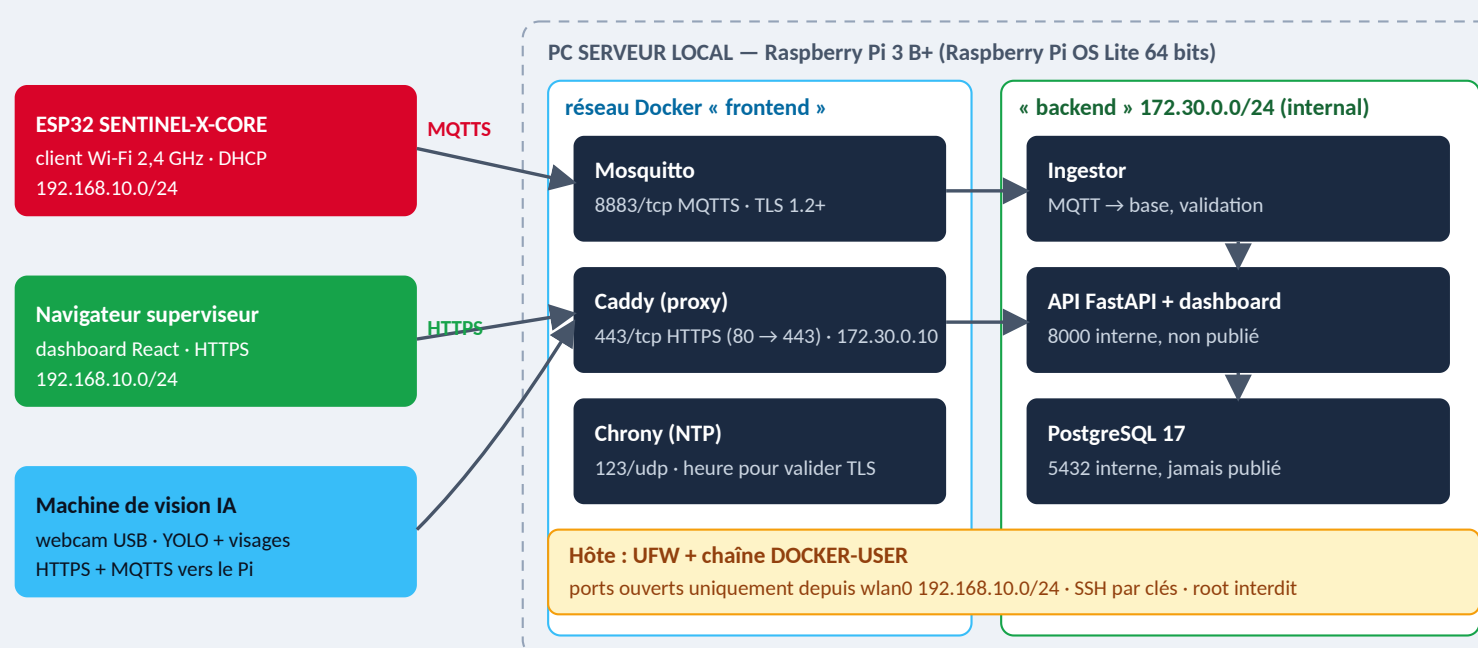
30 / 30

contrôles d'audit réussis

≈ 93 Mo

RAM mesurée des conteneurs

Architecture de la table



Plan d'adressage

wlan0 192.168.10.1/24 : point d'accès Wi-Fi de la table (hostapd, DHCP/DNS dnsmasq, « sentinel.local »), isolé des autres groupes
 eth0 : administration SSH (limitée en débit) · 10.42.0.0/24 : câble point à point Pi ↔ PC pour le flux caméra TCP 5001
 Docker : frontend (ports publiés) · backend 172.30.0.0/24 interne, sans route vers l'extérieur · aucun port 1883 ni 5432 publié

Sécurité en profondeur

- Flux IoT chiffrés de bout en bout : ECDSA P-256, CA épinglée dans le firmware, TLS 1.1 et anonyme refusés.
- Moindre privilège : un compte MQTT par composant, ACL au topic près, l'ESP32 ne peut pas envoyer de commande.
- Conteneurs non-root, `cap_drop ALL`, lecture seule, images épinglées.
- Badge RFID : clés dérivées par badge, signature HMAC, compteur anti-clonage, double vérification locale et serveur.
- Dashboard : rôles, 2FA TOTP, anti brute force, journal de sécurité append-only.

IA qui prévient avant l'incident

- Détection de présence YOLOv8n, dernière image uniquement pour une latence minimale.
- Visage inconnu confirmé sur 3 images sur 5 : alerte critique, alarme du robot, notification Discord.
- Isolation Forest sur 8 caractéristiques par fenêtre de 5 mesures : détecte la dérive lente d'un capteur.

Supervision temps réel

- Dashboard React : courbes à 1 s, flux vidéo annoté, jumeau numérique 3D de Wall-E animé par les actionneurs réels.
- Commandes à distance : sas, bras, tête, alarme, arrêt d'urgence.
- Alerte ressentie : halo d'écran, sirène, bandeau, onglet.

Preuves et documentation

- `scripts/audit-security.sh` : 30 contrôles · `scripts/smoke-test.sh` : 32 vérifications · 23 tests API.
- `docs/CONTRAT-MQTT.md`, `docs/API.md`, `docs/SECURITE.md`, `docs/AUDIT-SECURITE.md`.

Les quatre piliers

Edge & IoT

ESP32 : gaz MQ-2, DHT22, PIR, RFID RC522, OLED, 3 servos et moteur pas à pas. Firmware C++ `sentinel_core`.

IA locale

YOLOv8n (320 px, 5 fps), visages dlib 128 dimensions, Isolation Forest sans seuil statique.

Infrastructure

Docker-Compose : Mosquitto, PostgreSQL 17, ingestor, API FastAPI, Caddy HTTPS, NTP. Réseau backend interne.

Cybersécurité

TLS 1.2+, ACL par compte, comptes + 2FA, UFW + DOCKER-USER, SSH par clés, audit scripté.